

AIRRPrep: validated workflows for bulk and single-cell AIRR-seq preprocessing

Parsa Esmaili,¹ Fatemeh Chaker Hosseini Zavareh,¹ Nika Abdolahi¹ and Changiz Eslahchi¹

¹ComputImm, Tehran, Iran

* Corresponding author. fatemeh.ch.hosseini@gmail.com

Abstract

Motivation: Preprocessing adaptive immune receptor repertoire sequencing (AIRR-seq) data with established command-line tools requires local installation, scripting and knowledge of how interdependent steps must be ordered.

Results: We present AIRRPrep, a web application built on pRESTO and TRUST4. Three predefined bulk workflows implement documented pRESTO protocols, and a builder composes others from 47 pRESTO operations. Every workflow is checked against the uploaded files before it runs, and the same contracts enable or disable components while it is built; this establishes internal coherence, not the biological appropriateness of parameters. Workflow graphs, stepwise read retention and reusable workflow definitions are exportable. Single-cell inputs in seven formats are normalized into one internal FASTA and metadata representation without inventing cell identifiers, with the upstream annotations that the normalized schema omits written out unchanged beside it. On three public datasets AIRRPrep wrote the same records as command-line pRESTO at every step, and 46 of the 47 operations produced equivalent output when tested in isolation; the 47th could not be compared, because native pRESTO fails on that input. On one Windows 11 workstation, using 25,000-read-pair subsets, AIRRPrep completed the three workflows 1.2–2.8 times faster than command-line pRESTO; these measurements quantify small-input execution overhead rather than production-scale throughput.

Availability and Implementation: Free hosted service, no account required, at <https://preprocessing.computimm.com>; source code, container build and validation scripts at <https://github.com/ComputImm/AIRRPrep>, under the GNU Affero General Public License v3.0 or later; release v1.0.0 is archived at <https://doi.org/10.5281/zenodo.22857985>.

Contact: preprocessing@computimm.com

Supplementary information: Supplementary data are available at Bioinformatics online.

Keywords AIRR-seq, immune repertoire, preprocessing, pRESTO, TRUST4

Introduction

Raw adaptive immune receptor repertoire sequencing (AIRR-seq) reads must be preprocessed before germline annotation and analyses of clonality, lineage, diversity or selection. Depending on the library design this includes quality filtering, primer removal, extraction of unique molecular identifiers (UMIs), pairing and assembly of reads, UMI consensus construction and collapsing, and errors at this stage propagate downstream.

Existing tools implement these operations: pRESTO for read preprocessing (Vander Heiden et al., 2014), TRUST4 for receptor reconstruction from bulk and single-cell data (Song et al., 2021), nf-core/airrflow as an end-to-end pipeline (Gabernet et al., 2024), and the Immcantation container, which distributes pRESTO with downstream tools such as Change-O (Gupta et al., 2015). Using them requires command-line proficiency, local installation, familiarity with header conventions and knowledge of how interdependent steps must be ordered. Web platforms address broader scopes: VDJServer offers preprocessing, gene assignment, repertoire analysis and a data commons (Christley et al., 2018), iReceptor federates queries across

processed repertoires (Corrie et al., 2018), and ARGalaxy analyses and visualizes annotated repertoires in Galaxy (IJspeert et al., 2017). To our knowledge, none lets a user compose individual pRESTO operations interactively while checking the composition against the uploaded files before it runs (Supplementary Table S11).

AIRRPrep addresses the preprocessing stage itself. It exposes pRESTO and TRUST4 through a web application, separates bulk and single-cell data, checks every workflow against the uploaded files before execution, and makes each run inspectable and repeatable.

Implementation

Bulk workflows and pre-execution validation

AIRRPrep separates data at entry (Figure 1). Bulk input is one sequence file or matched Read 1 (R1) and Read 2 (R2) files, in FASTA or FASTQ. Three predefined workflows implement documented pRESTO reference protocols – UMI-barcoded MiSeq 2×250, non-UMI MiSeq 2×250 and UMI-barcoded 5' RACE MiSeq 325+275 –

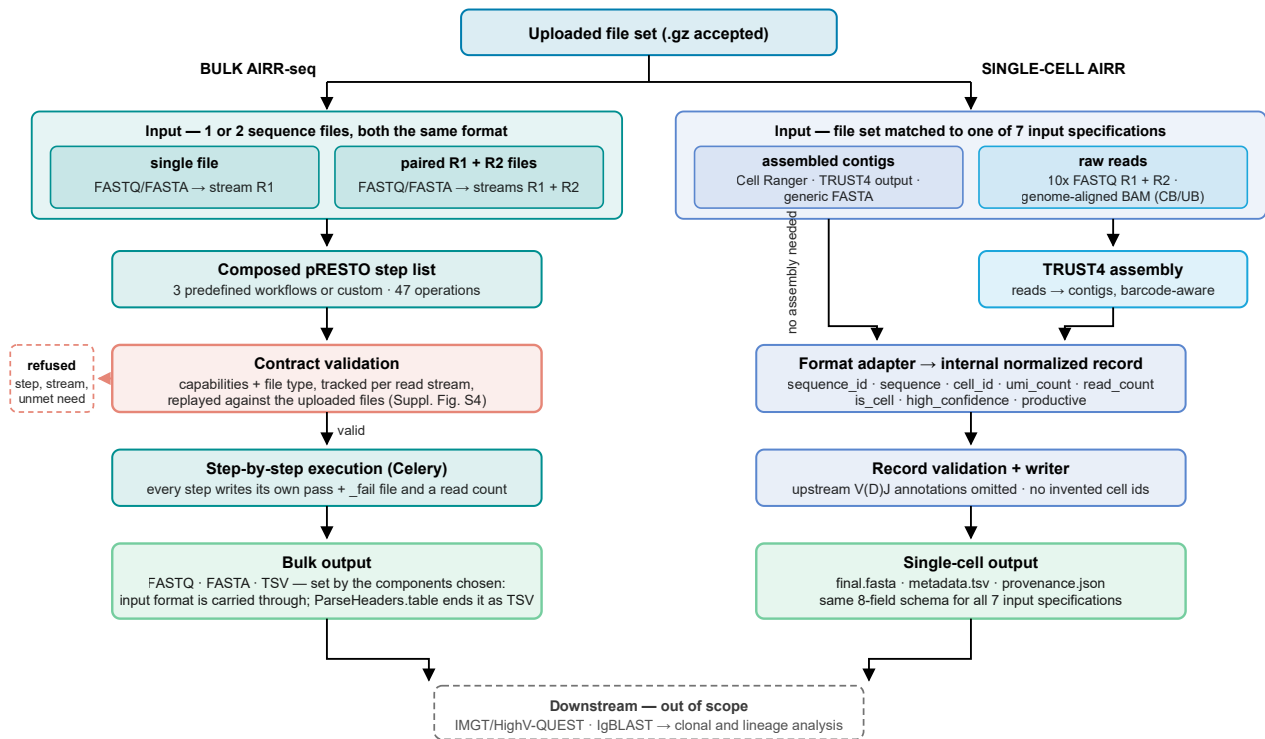


Figure 1 Overview of AIRRPrep. Bulk data are processed by predefined or custom pRESTO workflows validated before execution; single-cell inputs are converted directly or assembled with TRUST4, then normalized into AIRRPrep's internal FASTA and metadata representation, beside which the source's own annotation fields are written unchanged.

and a builder composes other designs from 47 operations across 13 pRESTO tools (Supplementary Table S2).

Each operation declares a contract: the file types and annotation capabilities it requires, the capabilities it produces, the companion files it needs, and whether it acts on one read stream, requires both, or merges them (Supplementary Figure S4). pRESTO checks the arguments of each command only when it runs, so on the command line an incoherent sequence of commands fails at the offending step, after earlier steps have run, or yields an empty file. AIRRPrep instead replays the workflow symbolically over the uploaded files before dispatch, tracking capabilities separately for R1 and R2, and refuses it at the first unmet contract, naming the step, the read stream and the missing requirement – for example consensus building before a barcode annotation exists, paired operations on single-file input, quality filtering on FASTA, or primer masking without a primer file. The same contracts are evaluated while a workflow is built: the selectable components are recomputed after every change, and a disabled component states what it lacks, so a valid pipeline can be assembled without knowing the correct order in advance (Supplementary Note S1).

The interface shows the workflow graph with the records entering and retained at each operation, for R1 and R2 separately and for the merged stream after assembly, so losses can be localized to a single step; the graphic and the counts are exportable (Supplementary Figures S2 and S3).

Single-cell normalization

Single-cell input is matched against seven input specifications: Cell Ranger all-contig or filtered-contig files, the Cell Ranger AIRR table, TRUST4 output, raw 10x FASTQ, a genome-aligned 10x BAM and generic receptor FASTA; incomplete file sets are reported with the files missing (Supplementary Table S1 and Figure S1). An adapter maps each format into an internal normalized record of eight fields – AIRRPrep's own schema, not an AIRR Rearrangement record – written as matching `final.fasta` and `metadata.tsv`, with values absent from the input left empty. Raw reads and BAM files are first assembled with TRUST4 in barcode-aware mode. A cell identifier is never invented: generic FASTA requires a barcode mapping file or header rule, a TRUST4 contig is assigned to a cell only through TRUST4's barcode report, and a BAM whose CB/UB tags cannot be used is refused rather than assembled without barcodes.

Upstream V(D)J calls, junctions and CDR3 annotations are excluded from the normalized record, so that sequences are re-annotated with one reference and one procedure, such as IMGT/HighV-QUEST (Alamyar et al., 2012). They are not discarded: every field the uploaded file carried for a record is written unchanged to a lossless sidebar, `source.annotations.tsv`, keyed on the same source `sequence_id`, so an inner join on that column restores the original annotation of any output record. The sidebar's own SHA-256, and that of every input, are recorded in `provenance.json`; recovering an omitted annotation therefore does not depend on the uploader still holding the original file (Supplementary Table S1).

Reproducibility and operation

A workflow, or a finished run, can be exported as a machine-readable definition and imported again. Uploaded companion files such as primer FASTA files are not embedded, because uploads expire; they are listed as required inputs with the name, size and SHA-256 checksum of the file originally used, and the definition records the software versions under which it was saved. No account is required: uploads belong to a browser session and are kept for ten days. Runtime is estimated before a job starts; long runs require a verified email address, and a six-digit tracking code sent by email restores the run from any browser when entered together with that address (Supplementary Note S2).

A React/TanStack Start frontend and a FastAPI backend dispatch jobs to Celery workers, with state held in Redis. The container image provides TRUST4, BLAST+, samtools, MUSCLE, VSEARCH and CD-HIT and can be deployed on institutional hardware; the proprietary USEARCH is not included, no predefined workflow requires it, and every default – including the clustering tool and the reference-assembly aligner – is a tool the image carries. Software versions are listed in Supplementary Table S10.

Validation

The three predefined workflows were run on the first 25,000 read pairs of SRR4026043 (Vander Heiden et al., 2017), ERR346600 (Greiff et al., 2014) and SRR1383456 (Stern et al., 2014) and compared step by step with equivalent command-line pRESTO scripts (Supplementary Table S9). At every step of all three, both implementations wrote the same records, with identical identifiers, sequences, quality strings and annotations; only record order differed (Supplementary Tables S3–S5). These comparisons cover 11 of the 47 operations.

Every operation was also tested on its own against the equivalent native pRESTO command, one archived case each. Of the 47 operations, 46 produced output equivalent to native pRESTO. **ConvertHeaders.py** **migec** could not be compared, because native pRESTO fails on multi-record MIGEC input, which AIRRPrep converts. The three **ClusterSets** cases of that matrix used VSEARCH, the open clustering tool the container image carries; in a separate, additional test with the proprietary USEARCH v7.0.1090, the command line native pRESTO constructs is rejected by USEARCH itself, whereas the invocation AIRRPrep constructs is accepted (Supplementary Table S2). The validator accepted the three predefined workflows and refused 14 invalid compositions spanning missing annotations, wrong file types, absent companion files, invalid read-stream states and impossible step orders (Supplementary Note S1).

Normalized single-cell output was compared record by record with three Cell Ranger representations of the 10x Genomics **sc5p_v2_hs_PBMC_10k** T-cell V(D)J data: all contigs (15,377 records), and filtered contigs and the AIRR table (10,817 each). None showed missing, additional or altered sequences, or mismatches in cell barcode, UMI or read count, cell, confidence or productive status, and every source annotation row joined back to its normalized record without collision or loss (Supplementary Table S6). Generic FASTA with a barcode mapping file or header rule reproduced the same records, and FASTA without a barcode rule was refused (Supplementary Table S7). The TRUST4 routes were run on the 10x **sc5p_v2_hs_PBMC_1k** B-cell library, from 598,163 raw read pairs and from its Cell Ranger BAM: in both, all 83 cells called by Cell Ranger were recovered with identical CDR3 nucleotide sequences at all 165 of their loci, no contig

was assigned an invented cell, uploading the resulting TRUST4 files reproduced the same 627 records, and unusable BAM files were refused. A genome-aligned gene-expression BAM of the same cells was also assembled correctly, with lower CDR3 agreement expected from its non-enriched coverage (Supplementary Table S7).

Execution time was compared over three replicates on one Windows 11 workstation with a four-core, eight-thread CPU, using the same Python and pRESTO installation; command-line pRESTO used its default of eight processes per parallel step, and AIRRPrep a single worker that never used more for the same step (Supplementary Tables S8 and S10). On that machine and on those 25,000-read-pair subsets, AIRRPrep was 1.21, 1.78 and 2.81 times faster. Calling pRESTO from a persistent worker removes per-step interpreter start-up, making operations outside read-pair assembly about three times faster (3.0–3.4×); that start-up cost is specific to Windows, which starts each worker process as a fresh interpreter where Linux forks, so the margin outside assembly should be expected to be smaller on a Linux host. Reference-guided assembly, which runs one single-threaded aligner per read pair in both implementations, took almost the same time (236 against 244 s) and accounts for 68% of the command-line runtime of the 5' RACE 325+275 workflow, hence that workflow's smaller gain. These figures quantify per-step execution overhead on small inputs on one Windows workstation; they are not a measurement of production-scale throughput, and no claim of platform-independent or production-scale advantage is made.

Discussion

AIRRPrep lowers the barrier to established AIRR-seq preprocessing and adds machine-checkable safeguards around workflow composition, both while a pipeline is built and before it runs; stepwise retention makes a run auditable, and an exported definition makes it repeatable. Validation establishes internal coherence, not biological appropriateness: a valid primer step can still use a wrong offset or primer set. The runtime margins are three-replicate means on 25,000-read-pair inputs on one Windows workstation, and characterize small-input execution overhead rather than throughput at production scale or on the Linux hosts a containerized deployment would normally use; assembly and TRUST4 reconstruction remain the expensive steps in both implementations. The scope is deliberately preprocessing: AIRRPrep does not assign germline genes, and its single-cell output is an internal record rather than an AIRR Rearrangement record (Vander Heiden et al., 2018), for which AIRR compliance is not claimed. Future work will add cross-step quality-control summaries, further single-cell platforms, a resource-matched Linux benchmark at production input sizes, and optional hand-off to annotation.

Data availability

AIRRPrep runs as a free hosted service at <https://preprocessing.computimm.com>. The source code of the frontend and the backend, the container build, the automated test suite, the validation scripts and their archived outputs (Supplementary Tables S2–S7) and the benchmark harness with every replicate time (Supplementary Tables S8–S9) are available at <https://github.com/ComputImm/AIRRPrep> under the GNU Affero General Public License v3.0 or later, the licence of pRESTO, which AIRRPrep builds on and partly redistributes. The version this manuscript describes is release v1.0.0

(commit 9148dba8bdd4c516b80db4eece07dd0a0b23a929), archived at <https://doi.org/10.5281/zenodo.22857985>. The bulky validation and benchmark outputs behind Supplementary Tables S2–S9 — the component matrix’s fixtures and per-case outputs, the single-cell and TRUST4 validation outputs and the benchmark run directories — are too large to hold in the repository and are archived separately at <https://doi.org/10.5281/zenodo.22858375>, with their SHA-256 checksums; `paper/EXPECTED_OUTPUTS.sha256` in the repository lists the checksum of every file inside them, so a re-run can be checked file by file without downloading the archives. The repository’s `README` maps each supplementary table to the script that produces it and to the result file it is read from.

The bulk validation data are the first 25,000 read pairs of SRR4026043 (Vander Heiden et al., 2017), ERR346600 (sample Replicate-1-M1) (Greiff et al., 2014) and SRR1383456 (Stern et al., 2014), whose sequences match the corresponding ENA FASTQ records one for one; the primer and reference FASTA files are those distributed with pRESTO for these protocols. The single-cell validation used the 10x Genomics Cell Ranger 5.0.0 outputs of `sc5p.v2.hs.PBMC.10k` and `sc5p.v2.hs.PBMC.1k` (<https://cf.10xgenomics.com/samples/cell-vdj/5.0.0/>); file checksums are listed in Supplementary Tables S6 and S7. USEARCH is proprietary and is not redistributed; the commands, version and outcome of the USEARCH-specific test are published, so that the test can be repeated by a reader holding a licence.

Author contributions

Parsa Esmaeili: Conceptualization, Software, Validation, Writing – original draft. **Fatemeh Chaker Hosseini Zavareh:** Software (lead), Methodology, Validation, Data curation, Writing – original draft, Writing – review & editing. **Nika Abdolahi:** Investigation, Validation, Writing – review & editing. **Changiz Eslahchi:** Conceptualization, Supervision, Writing – review & editing.

Funding

No specific funding was received for this work.

Conflict of interest

None declared.

References

- Vander Heiden JA, Yaari G, Uduman M, et al. pRESTO: a toolkit for processing high-throughput sequencing raw reads of lymphocyte receptor repertoires. *Bioinformatics*. 2014;30(13):1930–1932.
- Song L, Cohen D, Ouyang Z, Cao Y, Hu X, Liu XS. TRUST4: immune repertoire reconstruction from bulk and single-cell RNA-seq data. *Nature Methods*. 2021;18:627–630.
- Gabernet G, Marquez S, Bjornson R, et al. nf-core/airrflow: an adaptive immune receptor repertoire analysis workflow employing the Immcantation framework. *PLoS Computational Biology*. 2024;20(7):e1012265.
- Alamyar E, Duroux P, Lefranc M-P, Giudicelli V. IMGT/HighV-QUEST: the IMGT web portal for immunoglobulin and T

- cell receptor NGS repertoire analysis. *Immunome Research*. 2012;8:26.
- Gupta NT, Vander Heiden JA, Uduman M, Gadala-Maria D, Yaari G, Kleinstein SH. Change-O: a toolkit for analyzing large-scale B cell immunoglobulin repertoire sequencing data. *Bioinformatics*. 2015;31(20):3356–3358.
- Christley S, Scarborough W, Salinas E, et al. VDJServer: a cloud-based analysis portal and data commons for immune repertoire sequences and rearrangements. *Frontiers in Immunology*. 2018;9:976.
- Corrie BD, Marthandan N, Zimonja B, et al. iReceptor: a platform for querying and analyzing antibody/B-cell and T-cell receptor repertoire data across federated repositories. *Immunological Reviews*. 2018;284(1):24–41.
- IJspeert H, van Schouwenburg PA, van Zessen D, Pico-Knijnenburg I, Stubbs AP, van der Burg M. Antigen receptor Galaxy: a user-friendly, web-based tool for analysis and visualization of T and B cell receptor repertoire data. *The Journal of Immunology*. 2017;198(10):4156–4165.
- Vander Heiden JA, Marquez S, Marthandan N, et al. AIRR Community standardized representations for annotated immune repertoire data. *Frontiers in Immunology*. 2018;9:2206.
- Greiff V, Menzel U, Haessler U, et al. Quantitative assessment of the robustness of next-generation sequencing of antibody variable gene repertoires from immunized mice. *BMC Immunology*. 2014;15:40.
- Stern JNH, Yaari G, Vander Heiden JA, et al. B cells populating the multiple sclerosis brain mature in the draining cervical lymph nodes. *Science Translational Medicine*. 2014;6(248):248ra107.
- Vander Heiden JA, Stathopoulos P, Zhou JQ, et al. Dysregulation of B cell repertoire formation in myasthenia gravis patients revealed through deep sequencing. *The Journal of Immunology*. 2017;198(4):1460–1473.